Лабораторная работа №8 "Тестирование фронтенд-приложения"

Лабораторная работа №8. Тестирование фронтенд-приложения

1. ЦЕЛЬ РАБОТЫ

Освоение основ тестирования JavaScript кода и эффективного использования инструментов разработчика для отладки и анализа производительности веб-приложений **на примере вашего React-проекта, созданного в лабораторной работе №7**.

---

2. НЕОБХОДИМЫЕ ЗНАНИЯ

Для успешного выполнения работы студент должен знать:

- JavaScript ES6+ (функции, объекты, массивы, модули).
- Базовые понятия тестирования (unit-тесты, интеграционные тесты, покрытие).
- React компоненты и JSX (из лабораторной работы №7).
- HTML/CSS и структуру DOM.
- Инструменты разработчика браузера.

---

3. ЗАДАЧИ РАБОТЫ

- Настроить окружение для unit-тестирования с Jest и React Testing Library.
- Написать тесты для чистых JavaScript функций (утилит) из вашего проекта.
- Написать тесты для React компонентов (простых и композитных).
- Освоить инструменты разработчика для отладки (Console, Sources, Network, Performance).
- Научиться анализировать производительность с помощью Lighthouse.

---

4. ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ (общий для всех вариантов)

Часть 1. Настройка окружения для тестирования

1. **Перейдите в репозиторий вашего React-проекта** (из лабораторной работы №7) и создайте новую ветку:

```
git checkout main
git pull origin main
git checkout -b feature/frontend-testing
```

2. **Установите зависимости для тестирования** (если не установлены):

```
cd my-react-app   # папка вашего React-проекта
npm install --save-dev jest @testing-library/react @testing-library/jest-dom jest-environment-jsdom @testing-library/u
```

3. **Настройте скрипты в `package.json`:**

```
"scripts": {
  "test": "jest",
  "test:watch": "jest --watch",
  "test:coverage": "jest --coverage"
}
```

4. **Создайте конфигурационный файл `jest.config.js`** (опционально, но рекомендуется):

```
export default {
  testEnvironment: 'jsdom',
  setupFilesAfterEnv: ['<rootDir>/src/setupTests.js'],
  moduleNameMapper: {
    '\\.(css|less|scss|sass)$': 'identity-obj-proxy',
    '\\.(jpg|jpeg|png|gif|svg)$': '<rootDir>/__mocks__/fileMock.js'
  },
  collectCoverageFrom: [
    'src/**/*.{js,jsx}',
    '!src/**/*.test.{js,jsx}',
    '!src/main.jsx',
    '!src/vite-env.d.js'
  ],
  coverageThreshold: {
    global: {
      statements: 70,
      branches: 60,
      functions: 70,
      lines: 70
    }
  }
};
```

5. **Создайте файл `src/setupTests.js`:**

```
import '@testing-library/jest-dom';
```

Часть 2. Тестирование чистых JavaScript функций (утилит)

Объяснение: Начните с тестирования чистых функций, которые не зависят от React. Например, функции форматирования, валидации, преобразования данных.

Пример для утилит (создайте файл `src/utils/helpers.js` и `src/utils/helpers.test.js`):

```
// helpers.js
export function formatCurrency(amount, currency = 'RUB') {
    return new Intl.NumberFormat('ru-RU', { style: 'currency', currency }).format(amount);
}

export function truncateText(text, maxLength) {
    if (!text) return '';
    return text.length > maxLength ? text.slice(0, maxLength) + '...' : text;
}
```

```
// helpers.test.js
import { formatCurrency, truncateText } from './helpers';

describe('formatCurrency', () => {
    test('форматирует число в рубли', () => {
        expect(formatCurrency(1234.5)).toBe('1 234,50 ₽');
    });
    test('форматирует число в доллары', () => {
        expect(formatCurrency(99.99, 'USD')).toBe('99,99 $');
    });
});

describe('truncateText', () => {
    test('обрезает длинный текст', () => {
        expect(truncateText('Очень длинный текст для проверки', 10)).toBe('Очень длин...');
    });
    test('не обрезает короткий текст', () => {
        expect(truncateText('Короткий', 20)).toBe('Короткий');
    });
});
```

Задание для студента: Напишите unit-тесты для **минимум трёх** чистых функций, которые есть в вашем проекте (или создайте их специально). Функции должны быть связаны с вашей предметной областью (форматирование дат, расчёт скидки, фильтрация списка и т.п.).

Часть 3. Тестирование React компонентов

Объяснение: Используйте React Testing Library для тестирования компонентов, созданных в лабораторной работе №7.

Пример тестирования простого компонента (например, `Button.jsx`):

```
// Button.test.jsx
import { render, screen, fireEvent } from '@testing-library/react';
import Button from './Button';

describe('Button component', () => {
    test('рендерит кнопку с правильным текстом', () => {
        render(<Button>Нажми меня</Button>);
        expect(screen.getByText('Нажми меня')).toBeInTheDocument();
    });

    test('вызывает обработчик при клике', () => {
        const handleClick = jest.fn();
        render(<Button onClick={handleClick}>Клик</Button>);
        fireEvent.click(screen.getByText('Клик'));
        expect(handleClick).toHaveBeenCalledTimes(1);
    });
});
```

Задание для студента: Напишите тесты для **минимум трёх** ваших компонентов:

- Один базовый UI-компонент (например, `Button`, `Card`).
- Один компонент списка (например, `ProductList`, `NewsList`), проверяющий отображение данных из моковых массивов.
- Один компонент, содержащий обработчик событий (например, кнопка «добавить в корзину»), проверьте, что заглушка вызывается.

Важно: Так как вы не используете состояние, тестируйте только то, что компонент получает через `props` и какие функции-обработчики вызываются при взаимодействии.

Часть 4. Инструменты разработчика для отладки

Задание для студента: Выполните следующие шаги и задокументируйте их в отчёте.

1. **Console Tab:**
    - Добавьте в коде несколько `console.log`, `console.table`, `console.group` для отладки.
    - Сохраните скриншот вывода консоли.
2. **Sources Tab:**
    - Установите точку останова (breakpoint) в одном из ваших компонентов (например, в обработчике клика).
    - Выполните пошаговую отладку (Step Over, Step Into).
    - Наблюдайте за значениями переменных в панели Scope.
3. **Network Tab:**
    - Откройте вкладку Network, перезагрузите страницу.
    - Проанализируйте, какие ресурсы загружаются, найдите самые медленные запросы.
    - Сделайте скриншот.
4. **React Developer Tools (расширение для браузера):**
    - Установите расширение React DevTools.

- Изучите вкладку Components – проверьте props ваших компонентов.
- (Опционально) Вкладка Profiler – запишите профиль производительности.

Часть 5. Анализ производительности с Lighthouse

Задание для студента: Запустите аудит вашего React-приложения с помощью Lighthouse.

1. Откройте DevTools → вкладка Lighthouse.
2. Выберите категории: **Performance, Accessibility, Best Practices, SEO**.
3. Запустите генерацию отчёта.
4. Проанализируйте метрики:
   - **First Contentful Paint (FCP)** – время появления первого контента.
   - **Largest Contentful Paint (LCP)** – время загрузки самого большого элемента.
   - **Cumulative Layout Shift (CLS)** – стабильность макета.
5. Прочитайте рекомендации по улучшению (они появятся в отчёте).
6. Внесите **хотя бы одно** улучшение на основе рекомендаций (например, оптимизируйте изображение, добавьте `alt` атрибуты) и повторно запустите Lighthouse, чтобы увидеть изменение.

Часть 6. Запуск тестов и анализ покрытия

1. **Запустите тесты с отчётом о покрытии:**

```
npm run test:coverage
```

2. Откройте отчёт: `coverage/lcov-report/index.html` в браузере.
3. Проанализируйте, какие строки кода остались непокрытыми.
4. **Добавьте тесты**, чтобы достичь минимум 70% покрытия для утилит и 60% для компонентов (или выше).
5. Сделайте скриншот итогового отчёта о покрытии.

Часть 7. Фиксация изменений

```
git add .
git commit -m "feat: add unit tests, component tests, and performance audit with Lighthouse"
git push origin feature/frontend-testing
```

---

5. ИНДИВИДУАЛЬНЫЕ ВАРИАНТЫ ЗАДАНИЙ

Каждый студент работает со своим React-проектом, созданным в лабораторной работе №7.

- **Индивидуальное выполнение.** Все выполняют единый порядок действий (задание одинаково для всех вариантов, отличается только тематика проекта, которая выбирается в соответствии с будущим курсовым проектом).

  !!!! Для выбора темы используется Генератор вариантов

Общее задание для всех вариантов (единый порядок действий, с учётом вашей предметной области):

1. **Настройка тестового окружения** (Jest + React Testing Library).
2. **Unit-тесты для чистых функций:**
   - Реализуйте (или возьмите из проекта) 3–4 утилитарные функции, характерные для вашей тематики (например, форматирование даты, валюты, фильтрация списка, расчёт скидки).
   - Напишите тесты, покрывающие основные сценарии и граничные случаи.

3. **Тестирование компонентов:**
    - Протестируйте минимум 3 компонента (один простой UI, один композитный, один с обработчиком событий).
    - Используйте `render`, `screen`, `fireEvent` из `@testing-library/react`.
    - Проверьте, что компоненты правильно отображают переданные `props` и что колбэки вызываются.
4. **Отладка с помощью DevTools:**
    - Продемонстрируйте использование Console, Sources, Network, React DevTools (скриншоты).
5. **Анализ производительности:**
    - Запустите Lighthouse, запишите метрики, внесите минимум одно улучшение, покажите изменения.
6. **Покрытие кода:**
    - Добейтесь покрытия не менее 70% для утилит и 60% для компонентов (или общего покрытия не менее 65%).
    - Приложите скриншот отчёта о покрытии.

---

6. СОДЕРЖАНИЕ ОТЧЕТА

Отчёт должен содержать:

- Титульный лист (по Стандарту Предприятия).
- Цель работы.
- Краткие теоретические сведения (unit-тестирование, интеграционное тестирование, Jest, React Testing Library, Lighthouse).
- **Ход работы:**
    - Описание созданных чистых функций (код + тесты).
    - Описание протестированных компонентов (код + скриншоты тестов).
    - Скриншоты работы с DevTools (Console, Sources, Network, React DevTools).
    - Скриншоты отчёта Lighthouse до и после улучшений.
    - Скриншот отчёта о покрытии кода.
    - Ссылка на коммит с выполненной работой.
- **Ответы на контрольные вопросы** (все вопросы).
- **Выводы** (что узнали, какой опыт получили, как тестирование поможет в дальнейшей разработке).

---

7. ТРЕБОВАНИЯ К СДАЧЕ

- Отчёт (PDF) загружается в СЭО.
- **Индивидуальное выполнение.**
- Код должен быть загружен в репозиторий GitHub (ветка `feature/frontend-testing`).
- В README репозитория укажите:
    - Как запустить тесты (`npm test`).
    - Как сгенерировать отчёт о покрытии (`npm run test:coverage`).
    - Краткое описание проведённого анализа Lighthouse.

---

8. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Какие виды тестирования вы применили в работе? В чём разница между unit-тестами и интеграционными тестами?
2. Как Jest помогает в организации тестов? Какие функции Jest вы использовали (`describe`, `test`, `expect`, `jest.fn()` и др.)?
3. Что такое моки (mock) и зачем они нужны? Приведите пример из вашего проекта.
4. Что делает функция `render` из `@testing-library/react` и какие методы для поиска элементов вы использовали (`getByText`, `getByRole`, `queryBy...`)?
5. Как протестировать событие клика на кнопке? Напишите пример кода.
6. Какие инструменты разработчика оказались наиболее полезными при отладке? Опишите конкретную ситуацию, когда вы использовали breakpoints.
7. Какие метрики Lighthouse важны для пользовательского опыта? Что означают LCP, FID, CLS?
8. Как вы улучшили производительность своего приложения на основе рекомендаций Lighthouse? Приведите конкретный пример.
9. Что такое покрытие кода (code coverage) и почему оно важно? Какой процент покрытия вы достигли?
10. Как вы анализировали непокрытые участки кода? Что вы сделали, чтобы их покрыть тестами?
11. В чём разница между `getBy...` и `queryBy...` в React Testing Library? Когда использовать каждый?
12. Как протестировать компонент, который получает массив данных через `props` и рендерит список элементов?
13. Почему в тестах React компонентов важно использовать `data-testid` вместо поиска по тексту или классу? В каких случаях это необходимо?
14. Как проверить, что компонент вызвал функцию-обработчик, переданную в `props` (например, `onAddToCart`)? Приведите пример.
15. Какие рекомендации по доступности (accessibility) вы получили от Lighthouse и как их исправили?

---

9. ПРАКТИЧЕСКИЕ РЕКОМЕНДАЦИИ

- Перед написанием тестов убедитесь, что ваш React-проект собирается и работает (`npm run dev`).
- Для тестирования компонентов используйте `screen` для поиска элементов – это делает тесты читаемее.
- Для асинхронных операций (если они появятся) используйте `waitFor` или `findBy...`.
- Не забывайте очищать моки после каждого теста: `jest.clearAllMocks()`.
- Для анализа покрытия полезно запускать `npm run test:coverage` и открывать HTML-отчёт.

---

10. ПРИМЕЧАНИЯ

- Работа выполняется **индивидуально**.
- Это последняя лабораторная работа первой части курса. После её выполнения у вас должен быть полноценный React-проект с базовой структурой, стилями, моковыми данными, компонентами и набором тестов.
- Полученные навыки тестирования пригодятся при разработке курсового проекта во второй части дисциплины.